

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №10
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Поиск расстояний во взвешенном
графе»

Выполнил:

студенты группы 24BBB2

Бауэр А.А.

Гусев Е.В.

Медведев Е.А.

Принял:

к.т.н. доцент Юрова О.В.

к.т.н. Деев М.В.

Пенза 2025

Название

Поиск расстояний во взвешенном графе.

Цель работы

Научиться искать расстояния во взвешенном графе.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного взвешенного графа G . Выведите матрицу на экран
2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++
3. *Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2

1. Для каждого из вариантов сгенерированных графов (ориентированного и не ориентированного) определите радиус и диаметр.
2. Определите подмножества периферийных и центральных вершин.

Задание 3*

1. *Модернизируйте программу так, чтобы получить возможность запуска программы с параметрами командной строки (см. описание ниже). В качестве параметра должны указываться тип графа (взвешенный или нет) и наличие ориентации его ребер (есть ориентация или нет).

Описание метода решения задачи

Задание 1-2

Метод решения основан на модифицированном алгоритме поиска в ширину (BFS) для взвешенных графов. В отличие от классического BFS, где расстояние до каждой новой вершины увеличивается на единицу, в данной реализации расстояние вычисляется как сумма весов пройденных рёбер. Алгоритм начинает обход из стартовой вершины, помещает её в очередь и устанавливает расстояние до неё равным нулю. На каждом шаге из очереди извлекается текущая вершина, просматриваются все смежные с ней вершины. Для каждой непосещённой смежной вершины расстояние вычисляется как расстояние до текущей вершины плюс вес соединяющего их ребра, после чего эта вершина добавляется в очередь. Таким образом находятся кратчайшие расстояния от начальной вершины до всех остальных в смысле минимальной суммы весов рёбер. На основе этих расстояний вычисляются эксцентриситеты вершин, радиус и диаметр графа, а также определяются центральные и периферийные вершины.

Задание 3

Программа работает аналогично предыдущей. Единственное отличие-адаптация под консоль. Программа поддерживает два режима работы: интерактивный с меню выбора операций и неинтерактивный через параметры командной строки, что позволяет гибко настраивать тип графа и выполняемые действия.

Листинг

Файл 10LR.cpp (Задание 1-2)

```
#include <iostream>
#include <vector>
#include <queue>
#include <random>
#include <iomanip>
#include <limits>
#include <algorithm>
using namespace std;

// Класс для работы со взвешенными графами
class WeightedGraph {
private:
    vector<vector<int>>> adjacencyMatrix; // Матрица смежности
    int vertices; // Количество вершин
    bool isDirected; // Ориентированный или нет
    // Поиск расстояний с помощью BFS
    vector<int> bfsDistances(int startVertex) {
        if (startVertex < 0 || startVertex >= vertices) {
            return vector<int>(vertices, -1);
        }
        vector<int> dist(vertices, -1);
        queue<int> q;
        dist[startVertex] = 0;
        q.push(startVertex);
        while (!q.empty()) {
            int current = q.front();
            q.pop();

            for (int i = 0; i < vertices; i++) {
                if (adjacencyMatrix[current][i] > 0 && dist[i] == -
1) {
                    dist[i] = dist[current] +
adjacencyMatrix[current][i];
                    q.push(i);
                }
            }
        }
        return dist;
    }
public:
    // Конструктор - создает граф с заданным количеством вершин
    WeightedGraph(int n, bool directed = false) : vertices(n),
isDirected(directed) {
        generateRandomGraph();
    }
};
```

```

}
//Генерация случайного графа
void generateRandomGraph() {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> weightDis(1, 10);
    uniform_int_distribution<> edgeDis(0, 2);
    //Полностью пересоздаем матрицу с новым размером
    adjacencyMatrix.clear();// Очищаем старую матрицу
    adjacencyMatrix.resize(vertices, vector<int>(vertices, 0));

    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            if (i == j) continue;
            if (!isDirected && i < j) {
                if (edgeDis(gen) > 0) {
                    int weight = weightDis(gen);
                    adjacencyMatrix[i][j] = weight;
                    adjacencyMatrix[j][i] = weight;
                }
            }
            else if (isDirected) {
                if (edgeDis(gen) > 0) {
                    adjacencyMatrix[i][j] = weightDis(gen);
                }
            }
        }
    }
}

//Вывод матрицы смежности
void printAdjacencyMatrix() {
    cout << "\nМатрица смежности " << (isDirected ?
"ориентированного" : "неориентированного") << " графа:\n";
    cout << " ";
    for (int i = 0; i < vertices; i++) {
        cout << setw(4) << i;
    }
    cout << "\n";
    for (int i = 0; i < vertices; i++) {
        cout << setw(3) << i;
        for (int j = 0; j < vertices; j++) {
            cout << setw(4) << adjacencyMatrix[i][j];
        }
        cout << "\n";
    }
}

//Поиск расстояний от вершины
void findDistances(int startVertex) {
    vector<int> dist = bfsDistances(startVertex);
    cout << "Расстояния от вершины " << startVertex << ":\n";
    for (int i = 0; i < vertices; i++) {
        cout << "Вершина " << i << ": ";
        if (dist[i] == -1) {
            cout << "недостижима";
        }
    }
}

```

```

        else {
            cout << dist[i];
        }
        cout << "\n";
    }
}
//Вычисление эксцентриситета вершины
int calculateEccentricity(int vertex) {
    if (vertex < 0 || vertex >= vertices) {
        return -1;
    }
    vector<int> dist = bfsDistances(vertex);
    int eccentricity = -1;
    for (int i = 0; i < vertices; i++) {
        if (dist[i] != -1 && dist[i] > eccentricity) {
            eccentricity = dist[i];
        }
    }
    return eccentricity;
}
//Анализ графа - радиус, диаметр, центры
void analyzeGraph() {
    vector<int> eccentricities(vertices);
    int radius = numeric_limits<int>::max();
    int diameter = -1;
    //Считаем эксцентриситеты всех вершин
    for (int i = 0; i < vertices; i++) {
        eccentricities[i] = calculateEccentricity(i);
        if (eccentricities[i] != -1) {
            radius = min(radius, eccentricities[i]);
            diameter = max(diameter, eccentricities[i]);
        }
    }
    //Находим центральные и периферийные вершины
    vector<int> centralVertices, peripheralVertices;
    for (int i = 0; i < vertices; i++) {
        if (eccentricities[i] == radius)
            centralVertices.push_back(i);
        if (eccentricities[i] == diameter)
            peripheralVertices.push_back(i);
    }
    //Вывод результатов
    cout << "Эксцентриситеты вершин:\n";
    for (int i = 0; i < vertices; i++) {
        cout << "Вершина " << i << ": " << eccentricities[i] <<
"\n";
    }
    cout << "\nРадиус графа: " << radius << endl;
    cout << "Диаметр графа: " << diameter << endl;
    cout << "Центральные вершины: ";
    for (int v : centralVertices) cout << v << " ";
    cout << endl;
    cout << "Периферийные вершины: ";
    for (int v : peripheralVertices) cout << v << " ";
    cout << endl;
}

```

```

//Геттеры и сеттеры
int getVerticesCount() const { return vertices; }
bool getIsDirected() const { return isDirected; }
void setVertices(int n) {
    vertices = n;
    generateRandomGraph();
}
void setDirected(bool directed) { isDirected = directed;
generateRandomGraph(); }
void regenerateGraph() { generateRandomGraph(); }
};

int main() {
    setlocale(LC_ALL, "Russian");
    int vertices;
    cout << "Введите количество вершин графа: ";
    cin >> vertices;
    if (vertices <= 0) {
        cout << "Неверное количество вершин! Установлено значение по
умолчанию: 6" << endl;
        vertices = 6;
    }
    WeightedGraph graph(vertices, false);
    int choice;
    do {
        cout << "\n1. Неориентированный взвешенный граф";
        cout << "\n2. Ориентированный взвешенный граф";
        cout << "\n3. Анализ графа";
        cout << "\n4. Изменить количество вершин";
        cout << "\n0. Выход";
        cout << "\nВыберите операцию: ";
        cin >> choice;

        int startVertex, newVertices; // Выносим все объявления
переменных

        switch (choice) {
        case 1:
            if (graph.getIsDirected()) {
                graph.setDirected(false);
            }
            graph.printAdjacencyMatrix();
            cout << "Введите стартовую вершину для поиска
расстояний: ";
            cin >> startVertex;
            if (startVertex >= 0 && startVertex <
graph.getVerticesCount()) {
                graph.findDistances(startVertex);
            }
            else {
                cout << "Неверная вершина! Допустимый диапазон: 0-"
<< graph.getVerticesCount() - 1 << endl;
            }
            break;
        case 2:
            if (!graph.getIsDirected()) {

```

```

        graph.setDirected(true);
    }
    graph.printAdjacencyMatrix();
    cout << "Введите стартовую вершину для поиска
расстояний: ";
    cin >> startVertex;
    if (startVertex >= 0 && startVertex <
graph.getVerticesCount()) {
        graph.findDistances(startVertex);
    }
    else {
        cout << "Неверная вершина! Допустимый диапазон: 0-"
<< graph.getVerticesCount() - 1 << endl;
    }
    break;
case 3:
    cout << "\nАнализ текущего графа:" << endl;
    graph.printAdjacencyMatrix();
    graph.analyzeGraph();
    break;
case 4:
    cout << "Введите новое количество вершин: ";
    cin >> newVertices;
    if (newVertices > 0) {
        graph.setVertices(newVertices);
        graph.printAdjacencyMatrix();
    }
    else {
        cout << "Неверное количество вершин!" << endl;
    }
    break;
case 0:
    break;

default:
    cout << "Неверный выбор!" << endl;
}

} while (choice != 0);
return 0;
}

```

Файл 10LR_RE.cpp (Задание 3)

```

#include <iostream>
#include <vector>
#include <queue>
#include <random>
#include <iomanip>
#include <limits>
#include <algorithm>
#include <string>
#include <cstring>
using namespace std;

//Класс для работы со взвешенными графами
class WeightedGraph {

```

```

private:
    vector<vector<int>> adjacencyMatrix; // Матрица смежности графа
    int vertices; // Количество вершин в графе
    bool isDirected; // Флаг ориентированности графа
    // Поиск кратчайших расстояний от начальной вершины
    vector<int> bfsDistances(int startVertex) {
        // Добавляем проверку входных данных
        if (startVertex < 0 || startVertex >= vertices) {
            return vector<int>(vertices, -1);
        }

        vector<int> dist(vertices, -1); // Вектор расстояний, -1 -
недостижима
        queue<int> q; // Очередь для BFS обхода
        dist[startVertex] = 0; // Расстояние до самой себя равно 0
        q.push(startVertex); // Добавляем начальную вершину в очередь
        // Пока очередь не пуста, продолжаем обход
        while (!q.empty()) {
            int current = q.front(); // Берем первую вершину из
очереди

            q.pop(); // Удаляем ее из очереди
            // Проверяем всех соседей текущей вершины
            for (int i = 0; i < vertices; i++) {
                // Если есть ребро и вершина еще не посещена
                if (adjacencyMatrix[current][i] > 0 && dist[i] == -
1) {
                    // Обновляем расстояние до вершины i
                    dist[i] = dist[current] +
adjacencyMatrix[current][i];
                    q.push(i); // Добавляем вершину в очередь для
дальнейшего обхода
                }
            }
        }
        return dist; // Возвращаем вектор расстояний
    }

public:
    // Конструктор: создает граф с n вершинами
    WeightedGraph(int n, bool directed = false) : vertices(n),
isDirected(directed) {
        generateRandomGraph(); // Генерируем случайный граф
    }
    // Генерация случайного взвешенного графа
    void generateRandomGraph() {
        random_device rd;
        mt19937 gen(rd());
        uniform_int_distribution<> weightDis(1, 10); // Распределение
весов 1-10
        uniform_int_distribution<> edgeDis(0, 2);
        // Инициализируем матрицу смежности нулями
        adjacencyMatrix.clear();
        adjacencyMatrix.resize(vertices, vector<int>(vertices, 0));
        // Заполняем матрицу смежности
        for (int i = 0; i < vertices; i++) {
            for (int j = 0; j < vertices; j++) {

```



```

        if (i == j) continue;
        if (!isDirected && i < j) {
            //Для неориентированного графа обрабатываем
только верхний треугольник
            if (edgeDis(gen) > 0) {
                int weight = weightDis(gen); // Генерируем
случайный вес
                adjacencyMatrix[i][j] = weight; //
Устанавливаем вес для ребра i->j
                adjacencyMatrix[j][i] = weight; // И для j->i
(симметрично)
            }
        }
        else if (isDirected) {
            //Для ориентированного графа обрабатываем все
пары вершин
            if (edgeDis(gen) > 0) {
                adjacencyMatrix[i][j] = weightDis(gen); //
Устанавливаем вес для ребра i->j
            }
        }
    }
}
//Вывод матрицы смежности на экран
void printAdjacencyMatrix() {
    cout << "\nМатрица смежности " << (isDirected ?
"ориентированного" : "неориентированного") << " графа:\n";
    cout << " ";
    for (int i = 0; i < vertices; i++) {
        cout << setw(4) << i;
    }
    cout << "\n";
    for (int i = 0; i < vertices; i++) {
        cout << setw(3) << i;
        for (int j = 0; j < vertices; j++) {
            cout << setw(4) << adjacencyMatrix[i][j];
        }
        cout << "\n";
    }
}
//Поиск и вывод расстояний от заданной вершины до всех остальных
void findDistances(int startVertex) {
    vector<int> dist = bfsDistances(startVertex); // Получаем
расстояния через BFS
    cout << "Расстояния от вершины " << startVertex << ":\n";
    for (int i = 0; i < vertices; i++) {
        cout << "Вершина " << i << ": ";
        if (dist[i] == -1) {
            cout << "недостижима";
        }
        else {
            cout << dist[i];
        }
        cout << "\n";
    }
}

```

```

    }
    //Вычисление эксцентриситета вершины (максимальное расстояние до
других вершин)
    int calculateEccentricity(int vertex) {
        if (vertex < 0 || vertex >= vertices) {
            return -1;
        }

        vector<int> dist = bfsDistances(vertex); // Получаем
расстояния от вершины
        int eccentricity = -1; // Начальное значение эксцентриситета
        //Находим максимальное расстояние среди достижимых вершин
        for (int i = 0; i < vertices; i++) {
            if (dist[i] != -1 && dist[i] > eccentricity) {
                eccentricity = dist[i];
            }
        }
        return eccentricity;
    }
    //Полный анализ графа: радиус, диаметр, центральные и
периферийные вершины
    void analyzeGraph() {
        vector<int> eccentricities(vertices); // Вектор для хранения
эксцентриситетов всех вершин
        int radius = numeric_limits<int>::max(); // Радиус графа
(минимальный эксцентриситет)
        int diameter = -1; // Диаметр графа (максимальный
эксцентриситет)
        //Вычисляем эксцентриситеты всех вершин
        for (int i = 0; i < vertices; i++) {
            eccentricities[i] = calculateEccentricity(i);
            if (eccentricities[i] != -1) {
                radius = min(radius, eccentricities[i]); // Обновляем
радиус
                diameter = max(diameter, eccentricities[i]); //
Обновляем диаметр
            }
        }
        //Находим центральные и периферийные вершины
        vector<int> centralVertices, peripheralVertices;
        for (int i = 0; i < vertices; i++) {
            if (eccentricities[i] == radius) {
                centralVertices.push_back(i); // Вершины с
эксцентриситетом = радиусу
            }
            if (eccentricities[i] == diameter) {
                peripheralVertices.push_back(i); // Вершины с
эксцентриситетом = диаметру
            }
        }
        //Вывод результатов анализа
        cout << "Эксцентриситеты вершин:\n";
        for (int i = 0; i < vertices; i++) {
            cout << "Вершина " << i << ": " << eccentricities[i] <<
"\n";
        }
    }

```

```

    cout << "\nРадиус графа: " << radius << endl;
    cout << "Диаметр графа: " << diameter << endl;
    cout << "Центральные вершины: ";
    for (int v : centralVertices) cout << v << " ";
    cout << endl;
    cout << "Периферийные вершины: ";
    for (int v : peripheralVertices) cout << v << " ";
    cout << endl;
}
//Геттеры для доступа к приватным полям
int getVerticesCount() const { return vertices; }
bool getIsDirected() const { return isDirected; }
//Сеттеры для изменения параметров графа
void setVertices(int n) { vertices = n; generateRandomGraph(); }
void setDirected(bool directed) { isDirected = directed;
generateRandomGraph(); }
};

//Обработка параметров командной строки
void processCommandLine(int argc, char* argv[], int& vertices, bool&
isDirected, bool& doAnalysis, int& startVertex, bool&
interactiveMode) {
    // Устанавливаем значения по умолчанию
    vertices = 6;
    isDirected = false;
    doAnalysis = false;
    startVertex = -1;
    interactiveMode = true;
    //Если нет параметров - используем интерактивный режим
    if (argc == 1) {
        return;
    }
    //Обрабатываем переданные параметры
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-v") == 0 && i + 1 < argc) {
            vertices = atoi(argv[i + 1]); // Количество вершин
            i++; // Пропускаем следующий аргумент (значение)
        }
        else if (strcmp(argv[i], "-d") == 0) {
            isDirected = true; // Ориентированный граф
        }
        else if (strcmp(argv[i], "-u") == 0) {
            isDirected = false; // Неориентированный граф
        }
        else if (strcmp(argv[i], "-a") == 0) {
            doAnalysis = true; // Выполнить анализ
            interactiveMode = false; // Переходим в неинтерактивный
режим
        }
        else if (strcmp(argv[i], "-s") == 0 && i + 1 < argc) {
            startVertex = atoi(argv[i + 1]); // Стартовая вершина
            interactiveMode = false; // Переходим в неинтерактивный
режим
            i++; // Пропускаем следующий аргумент (значение)
        }
    }
}

```

```

//Если указаны параметры для неинтерактивного режима
if (doAnalysis || startVertex != -1) {
    interactiveMode = false;
}
}

int main(int argc, char* argv[]) {
    setlocale(LC_ALL, "Russian");
    //Переменные для хранения параметров
    int vertices;
    bool isDirected;
    bool doAnalysis;
    int startVertex;
    bool interactiveMode;
    //Обрабатываем параметры командной строки
    processCommandLine(argc, argv, vertices, isDirected, doAnalysis,
startVertex, interactiveMode);
    //Проверяем корректность количества вершин
    if (vertices <= 0) {
        cout << "Неверное количество вершин! Установлено значение по
умолчанию: 6" << endl;
        vertices = 6;
    }
    //Создаем граф с заданными параметрами
    WeightedGraph graph(vertices, isDirected);
    //Неинтерактивный режим (если указаны параметры командной
строки)
    if (!interactiveMode) {
        graph.printAdjacencyMatrix();// Всегда выводим матрицу
смежности
        //Если запрошен анализ графа
        if (doAnalysis) {
            graph.analyzeGraph();
        }
        //Если указана стартовая вершина для поиска расстояний
        if (startVertex != -1) {
            if (startVertex >= 0 && startVertex <
graph.getVerticesCount()) {
                graph.findDistances(startVertex);
            }
            else {
                cout << "Неверная стартовая вершина: " <<
startVertex << endl;
            }
        }
        return 0;
    }
    //Интерактивный режим
    int choice;
    int newVertices;
    do {
        cout << "\n1. Неориентированный взвешенный граф";
        cout << "\n2. Ориентированный взвешенный граф";
        cout << "\n3. Анализ графа";
        cout << "\n4. Изменить количество вершин";
        cout << "\n0. Выход";
    }

```

```

cout << "\nВыберите операцию: ";
cin >> choice;
switch (choice) {
case 1:
    if (graph.getIsDirected()) {
        graph.setDirected(false); // Устанавливаем
неориентированный граф только если нужно
    }
    graph.printAdjacencyMatrix(); // Выводим матрицу
    cout << "Введите стартовую вершину для поиска
расстояний: ";
    cin >> startVertex;
    if (startVertex >= 0 && startVertex <
graph.getVerticesCount()) {
        graph.findDistances(startVertex); // Ищем расстояния
    }
    else {
        cout << "Неверная вершина! Допустимый диапазон: 0-"
<< graph.getVerticesCount() - 1 << endl;
    }
    break;
case 2:
    if (!graph.getIsDirected()) {
        graph.setDirected(true); // Устанавливаем
ориентированный граф только если нужно
    }
    graph.printAdjacencyMatrix(); // Выводим матрицу
    cout << "Введите стартовую вершину для поиска
расстояний: ";
    cin >> startVertex;
    if (startVertex >= 0 && startVertex <
graph.getVerticesCount()) {
        graph.findDistances(startVertex); // Ищем расстояния
    }
    else {
        cout << "Неверная вершина! Допустимый диапазон: 0-"
<< graph.getVerticesCount() - 1 << endl;
    }
    break;
case 3:
    graph.printAdjacencyMatrix(); // Выводим матрицу
    graph.analyzeGraph(); // Выполняем анализ графа
    break;
case 4:
    cout << "Введите новое количество вершин: ";
    cin >> newVertices;
    if (newVertices > 0) {
        graph.setVertices(newVertices); // Изменяем
количество вершин
        graph.printAdjacencyMatrix(); // Выводим новую
матрицу
    }
    else {
        cout << "Неверное количество вершин!" << endl;
    }
    break;

```

```

        case 0:
            break;
        default:
            cout << "Неверный выбор!" << endl;
    }
} while (choice != 0);
return 0;
}

```

Текст к программе

1. 10LR.cpp (Задание 1-2)

Данная программа реализует анализ взвешенных графов с использованием модифицированного алгоритма поиска в ширину. Пользователь взаимодействует с программой через консольное меню, где может выбрать тип графа (ориентированный или неориентированный), выполнить поиск расстояний от указанной вершины, провести полный анализ графа или изменить количество вершин. Программа автоматически генерирует случайный граф с заданными параметрами, где веса рёбер распределяются в диапазоне от 1 до 10. Основной особенностью является адаптация классического BFS для работы со взвешенными графами - вместо увеличения расстояния на 1 на каждом шаге, алгоритм суммирует веса рёбер при обходе графа.

2. 10LR_RE.cpp (Задание 3)

Эта программа представляет собой расширенную версию базовой реализации с добавлением поддержки параметров командной строки. Она сохраняет все функции предыдущей версии, но предоставляет два режима работы: интерактивный (через меню) и неинтерактивный (через параметры командной строки). Пользователь может указать тип графа, количество вершин, необходимость выполнения анализа и стартовую вершину непосредственно при запуске программы. Это позволяет автоматизировать процесс работы с графами и интегрировать программу в скрипты обработки данных. Оба режима используют одинаковый алгоритмический подход для вычисления расстояний и анализа графовых характеристик.

Результаты работы программы

Результаты работы программы показаны на рисунках 2-3.

I. 10LR.cpp (1-2 задание)

```

Введите количество вершин графа: 5

1. Неориентированный взвешенный граф
2. Ориентированный взвешенный граф
3. Анализ графа
4. Изменить количество вершин
0. Выход
Выберите операцию: 1

Матрица смежности неориентированного графа:
      0   1   2   3   4
0     0   7   2   5   0
1     7   0   1  10  10
2     2   1   0   9   9
3     5  10   9   0   3
4     0  10   9   3   0

Введите стартовую вершину для поиска расстояний: 0
Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 7
Вершина 2: 2
Вершина 3: 5
Вершина 4: 17

```

Рисунок 1 – Вывод матрицы смежности неориентированного взвешенного графа + вывод расстояний от 0 вершины

```

1. Неориентированный взвешенный граф
2. Ориентированный взвешенный граф
3. Анализ графа
4. Изменить количество вершин
0. Выход
Выберите операцию: 2

Матрица смежности ориентированного графа:
      0   1   2   3   4
0     0  10   3   4   3
1     4   0   4   9   5
2     9   9   0   8   9
3     6   3   9   0   1
4     2   3   6   7   0

Введите стартовую вершину для поиска расстояний: 0
Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 10
Вершина 2: 3
Вершина 3: 4
Вершина 4: 3

```

Рисунок 2 – Вывод матрицы смежности ориентированного взвешенного графа + вывод расстояний от 0 вершины

```

Введите количество вершин графа: 5

1. Неориентированный взвешенный граф
2. Ориентированный взвешенный граф
3. Анализ графа
4. Изменить количество вершин
0. Выход
Выберите операцию: 3

Анализ текущего графа:

Матрица смежности неориентированного графа:
      0   1   2   3   4
0     0   0   9   8   5
1     0   0   3   6   3
2     9   3   0   1   0
3     8   6   1   0   6
4     5   3   0   6   0

Эксцентриситеты вершин:
Вершина 0: 12
Вершина 1: 12
Вершина 2: 14
Вершина 3: 8
Вершина 4: 14

Радиус графа: 8
Диаметр графа: 14
Центральные вершины: 3
Периферийные вершины: 2 4

```

Рисунок 3 – Анализ неориентированного взвешенного графа


```

1. Неориентированный взвешенный граф
2. Ориентированный взвешенный граф
3. Анализ графа
4. Изменить количество вершин
0. Выход
Выберите операцию: 3

Анализ текущего графа:

Матрица смежности ориентированного графа:
      0  1  2  3  4
0  0  6  5  5  2
1  0  0  7  7  3
2  5  5  0  1  0
3  8  6  6  0  8
4  4  2  0  6  0

Эксцентриситеты вершин:
Вершина 0: 6
Вершина 1: 12
Вершина 2: 7
Вершина 3: 8
Вершина 4: 9

Радиус графа: 6
Диаметр графа: 12
Центральные вершины: 0
Периферийные вершины: 1

```

Рисунок 4 – Анализ ориентированного взвешенного графа

```

1. Неориентированный взвешенный граф
2. Ориентированный взвешенный граф
3. Анализ графа
4. Изменить количество вершин
0. Выход
Выберите операцию: 4
Введите новое количество вершин: 12

Матрица смежности неориентированного графа:
      0  1  2  3  4  5  6  7  8  9  10  11
0  0  9  6  3  0  5  4  7  9  3  3  0
1  9  0  3  6  0  0  3  0  9  10  0  0
2  6  3  0  5  7  0  0  4  2  5  1  2
3  3  6  5  0  5  2  0  1  1  10  8  5
4  0  0  7  5  0  5  1  0  1  10  0  6
5  5  0  0  2  5  0  0  4  6  1  0  4
6  4  3  0  0  1  0  0  3  2  8  5  0
7  7  0  4  1  0  4  3  0  5  4  1  1
8  9  9  2  1  1  6  2  5  0  0  0  7
9  3  10  5  10  10  1  8  4  0  0  0  1
10  3  0  1  8  0  0  5  1  0  0  0  3
11  0  0  2  5  6  4  0  1  7  1  3  0

```

Рисунок 5 – Изменение размеров графа

```

1. Неориентированный взвешенный граф
2. Ориентированный взвешенный граф
3. Анализ графа
4. Изменить количество вершин
0. Выход
Выберите операцию: -5
Неверный выбор!

```

Рисунок 6 – Попытка ввода отрицательно числа

II. 10LR_RE.cpp (3 задание)

```

(с) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Admin>cd C:\Users\Admin\Desktop\Algorithmic Logic in Engineering\10LR_RE\x64\Debug
C:\Users\Admin\Desktop\Algorithmic Logic in Engineering\10LR_RE\x64\Debug>10LR_RE.exe

1. Неориентированный взвешенный граф
2. Ориентированный взвешенный граф
3. Анализ графа
4. Изменить количество вершин
0. Выход
Выберите операцию: 0

C:\Users\Admin\Desktop\Algorithmic Logic in Engineering\10LR_RE\x64\Debug>10LR_RE.exe -v 6 -d -a

Матрица смежности ориентированного графа:
  0  1  2  3  4  5
0  0  4  7  10  0  7
1  0  0  5  0  0  3
2  5  9  0  0  9  3
3  2  10  7  0  4  0
4  4  0  0  0  4  0
5  8  0  8  5  7  0

Эксцентриситеты вершин:
Вершина 0: 16
Вершина 1: 14
Вершина 2: 15
Вершина 3: 10
Вершина 4: 11
Вершина 5: 12

Радиус графа: 10
Диаметр графа: 16
Центральные вершины: 3
Периферийные вершины: 0

C:\Users\Admin\Desktop\Algorithmic Logic in Engineering\10LR_RE\x64\Debug>10LR_RE.exe -v 12

1. Неориентированный взвешенный граф
2. Ориентированный взвешенный граф
3. Анализ графа
4. Изменить количество вершин
0. Выход
Выберите операцию: 1

Матрица смежности неориентированного графа:
  0  1  2  3  4  5  6  7  8  9  10  11
0  0  7  10  10  9  3  3  10  0  8  1  8
1  7  0  1  7  3  3  1  1  0  4  7  8
2  10  1  0  9  5  1  0  2  1  9  3  3
3  10  7  9  0  5  5  8  2  3  9  7  9
4  9  3  5  5  0  8  10  3  3  5  0  4
5  3  3  1  5  8  0  8  4  10  7  1  8
6  3  1  0  8  10  8  0  0  9  4  7  0
7  10  1  2  2  3  4  0  0  3  1  6  7
8  0  0  1  3  3  10  9  3  0  5  1  7
9  8  4  9  9  5  7  4  1  5  0  7  8
10  1  7  3  7  0  1  7  6  1  7  0  0
11  8  8  3  9  4  8  0  7  7  8  0  0

Введите стартовую вершину для поиска расстояний: 0
Расстояния от вершины 0:
Вершина 0: 0
Вершина 1: 7
Вершина 2: 10

```

Рисунок 7 – Работа программы в консоли

Выводы

В ходе лабораторной работы был реализован алгоритм поиска расстояний во взвешенных графах на основе модифицированного обхода в ширину. Программа позволяет генерировать случайные ориентированные и неориентированные графы, находить кратчайшие пути и анализировать основные характеристики графа - радиус, диаметр, эксцентриситеты вершин. Реализация поддерживает два режима работы: интерактивный с меню выбора операций и пакетный через параметры командной строки. Тестирование подтвердило корректность работы алгоритма для графов различной структуры и размера.